

SEPM

Name: Yash Kadam

Roll no.: 62

Batch: T-13

CASE STUDY: Optimizing Software Delivery Through Dev-Ops

* INTRODUCTION:

In a traditional IT environment, a major conflict often exists between development and operations teams.

While Developers aim to introduce new features quickly to meet business needs, Operations teams focus on maintaining site stability and speed, often viewing frequent changes as a threat to that stability.

This case study explores how an organization can overcome this "Wall of Confusion" by adopting DevOps principles and practices

* THE CHALLENGE: "Wall of Confusion"

Before adopting DevOps, the organization faced significant inefficiencies:

- **Siloed Roles**: Developers were responsible for adding features, while operations handled system administration and security.
- **Disconnected Environments**: Inconsistent environments between development, testing and production led to the "it works on my machine" syndrome.
- **Manual Errors**: Manual infrastructure setup often involved "hacking" scripts, leading to productional issues.
- **Integration delays**: Code changes were only integrated at the end of the development cycle, making merging hard and unpredictable.

* DevOps PRINCIPLES: THE CALMS FRAMEWORK

To address these challenges, the organization implemented CALMS Framework, which defines the core philosophy of DevOps:

- Culture: Transitioning to a collaborative environment where DevOps is everyone's job, & not just one team's responsibility.
- Automation: Eliminating repetitive manual work to create reliable, repeatable systems.
- Learn: Embracing improvement and building systems for rapid recovery when failures occur.
- Measurement: Using metrics (eg lead-time, recovery time and failure rates) to guide decision making.
- Sharing: Pairing developers and operators throughout the application lifecycle to share successes and failures.

* DevOps PRACTICES AND TOOLS

The organization adopted several key practices to bridge the gap between development and operations:

① Continuous Development and Integration (CI)

Developers plan, code and manage builds using version control tools like Git and Github.

Every revision triggers an automated build and unit test to find bugs quickly.

Tools: Jenkins, Circle CI, Travis CI

② Continuous Testing:

Automated tests are executed repeatedly against the code base to ensure continuous quality.

Tools: Selenium, Playwright, Bamboo.

③ Continuous Delivery and Deployment (CD)

Only code proven to be "good" through automation becomes part of software product.

While Continuous Delivery requires a manual trigger to deploy the function to production, Continuous deployment automates this final step.

④ Infrastructure as Code (IaC)

Infrastructure and networking settings are described in text files, allowing for automated provisioning and consistent environments.

Tools: Terraform, Ansible, Kubernetes

* THE DEVOPS ENGINEER: Role and Responsibilities

To sustain these principles and practices, the organization established the role of DevOps Engineer acting as a Bridge between responsibilities:

Category	Details
Core Responsibilities	CI/CD management, infrastructure management, scripting/coding and performance monitoring
Technical Skills	Proficiency in Linux, cloud services and programming languages like Java, Python, Ruby.
Personal Attributes	Strong analytical skills, leadership, empathy, and the ability to work in cross-functional teams.

* CONCLUSION & BENEFITS

By adopting DevOps, the organization transformed its delivery process.

- Faster Time to Market: Increased productivity and expedited the software development lifecycle (SDLC)
- Enhanced Stability: Reduced development failures and achieved 24x faster recovery from system failures
- Improved Quality: High IT performance correlated with better business outcomes and higher user trust.